

GPTHub Prod — архитектура, сценарии, модели, зависимости (текст для PDF)

Поддерживаемые сценарии (единый чат Open WebUI)

- Текстовый диалог; голос (STT/TTS на стороне WebUI, далее тот же оркестратор).
- Вложения: PDF, Office (DOCX/XLSX/PPTX и др. через markdown), десятки текстовых форматов; картинка в сообщении (VLM); аудио с ASR; URL в тексте (fetch HTML, SSRF-защита, лимиты).
- Веб-поиск (Tavily через WebUI при включении; сниппеты в запрос с bypass векторизации при необходимости).
- Долгосрочная память (команды запомнить/забудь/список; retrieval в system-контекст).
- Генерация изображений по интену (MWS /v1/images/generations).
- WOW: Expert Council (глубокое исследование, fan-out + один синтез); WOW: PPTX (план, python-pptx, ссылка на артефакт).
- Наблюдаемость: заголовок X-GPTHub-Trace (маршрут, цепочка моделей, артефакты ingest).

Контур сервисов и моделей (кратко)

Пользователь → Open WebUI → orchestrator (ingest, classifier/router, memory, council, image-gen, PPTX, trace) → LiteLLM (алиасы) → MWS (чат, VLM, документы, reasoning). ASR, image-gen, эмбединги памяти — отдельные вызовы в MWS. Опционально: Tavily и embedding shim из WebUI (пунктир на Mermaid-схеме ниже).

User Flow (кратко)

Одно сообщение в тред → POST /v1/chat/completions (OpenAI-форма) → orchestrator: ingest/classify/route → ветка memory ИЛИ image ИЛИ pptx ИЛИ council ИЛИ обычный chat через LiteLLM → MWS → ответ пользователю + X-GPTHub-Trace. Все ветки остаются в одном продуктовом чате.

Модели и алиасы (см. infra/litellm/config.yaml, docs/MWS_CATALOG.md)

- Базовый чат: mws-gpt-alpha (gpt-hub-turbo); тяжёлый: glm-4.6-357b (gpt-hub-strong); документы: qwen2.5-72b-instruct (gpt-hub-doc); код/рассуждения: qwen3-coder-480b-a35b (gpt-hub-reasoning-or).
- VLM: цепочка gpt-hub-vision → vision-2 → vision-3 → vision-4 → fallback (qwen3-vl-*, qwen2.5-vl*, cotypro-pro-vl-32b и др.).
- Fallback текстовый: gemma-3-27b-it. Картинки: qwen-image. ASR: whisper-medium. Память (эмбединги): qwen3-embedding-8b. План PPTX: алиасы gpt-hub-pptx-* и др.

Внешние зависимости

- Обязательно: MWS GPT API (base/key в .env / .env.mws.local); Docker Compose для Open WebUI + orchestrator + LiteLLM.
- Локально для памяти: SQLite (факты). Без MWS основной чат не работает.
- Опционально: Tavily (TAVILY_API_KEY, ENABLE_WEB_SEARCH, BYPASS_WEB_SEARCH_EMBEDDING_AND_RETRIEVAL при отсутствии embedding engine); markdown (Office ingest); embedding-shim (профиль rag в compose).

Диаграммы

Ниже в PDF идут **две текстовые схемы** — полные файлы docs/submission/architecture.mmd и user_flow.mmd (Mermaid). Это и есть «схема» для конкурса: контур сервисов/моделей и User Flow по одному чату. Визуальный PNG для презентаций: mmdc (@mermaid-js/mermaid-cli), опционально.

Схема 1. Контур сервисов и моделей, поддерживающих единый чат (исходник Mermaid: architecture.mmd). Визуальный PNG при необходимости собирается отдельно командой mmdc — в этом PDF только текст.

```
--- architecture.mmd part 1/1 ---
%% GPTHub prod — service contour + side paths. Export to PNG for submission / slides.
%% mmdc: npx -y @mermaid-js/mermaid-cli -i docs/submission/architecture.mmd -o docs/submission/
architecture.png
flowchart TB
    subgraph ux [User]
        U[User_single_chat]
    end
    subgraph webui [OpenWebui]
        OW[OpenWebUI_UI]
    end
    subgraph orchLayer [Orchestrator_FastAPI]
        OR[Orchestrator]
        IN[Ingest_pipeline]
        MEM[Memory_SQLite_plus_MWS_embeddings]
        IMG[Image_gen_MWS]
        PPTX[PPTX_plan_build_artifacts]
        COUNCIL[Expert_Council_fanout]
        TR[X-GPTHub-Trace]
    end
    subgraph gateway [LiteLLM]
        LL[LiteLLM_alias_layer]
    end
    subgraph mws [MWS_upstream]
        M[MWS_models_chat_VLM_doc_reasoning]
        MW[whisper_ASR]
        MI[qwen_image]
        ME[qwen3_embedding]
    end
    subgraph ext [External_optional]
        TV[Tavily_via_WebUI_web_search]
        SH[Embedding_shim_optional_rag_profile]
    end
    U --> OW
    OW -->|"OpenAI_compatible_POST"| OR
    OR --> IN
    OR --> MEM
    OR --> IMG
    OR --> PPTX
    OR --> COUNCIL
    OR --> TR
    OR -->|"chat_completion_path"| LL
    LL --> M
    OR -->|"ASR_path"| MW
    IMG --> MI
    MEM --> ME
    OW --> TV
    OW --> SH
```

Схема 2. User Flow — путь пользователя по одному чату (исходник Mermaid: user_flow.mmd).
Ветки short-circuit и основной chat path.

```
--- user_flow.mmd part 1/1 ---
%% GPTHub prod — User Flow (single chat). Export to PNG for submission / slides.
%% mmdc: npx -y @mermaid-js/mermaid-cli -i docs/submission/user_flow.mmd -o docs/submission/
user_flow.png
sequenceDiagram
    participant User
    participant WebUI as OpenWebUI
    participant Orch as Orchestrator
    participant Lite as LiteLLM
    participant MWS as MWS_models
    User->>WebUI: Message_and_optional_attachments_same_thread
    WebUI->>Orch: POST_v1_chat_completions_OpenAI_shape
    Note over Orch: Ingest_files_URL_audio_classify_route
    alt Memory_command_short_circuit
        Orch-->>WebUI: Reply_without_LiteLLM
    else Image_intent_short_circuit
        Orch->>MWS: Images_generations
        Orch-->>WebUI: Markdown_inline_image_plus_trace
    else PPTX_intent_short_circuit
        Orch->>Lite: Slide_plan_models_alias
        Lite->>MWS: Plan_completion
        Orch-->>WebUI: Markdown_preview_plus_download_link_plus_trace
    else Expert_council_short_circuit
        Orch->>Lite: Parallel_expert_models
        Lite->>MWS: Fanout_and_synthesis
        Orch-->>WebUI: Single_synthesized_reply_plus_trace
    else Default_chat_path
        Orch->>Lite: Routed_alias_gpt-hub-asterisk
        Lite->>MWS: Upstream_chat_completion
        MWS-->>Lite: Tokens
        Lite-->>Orch: OpenAI_compatible_chunk_or_json
        Orch-->>WebUI: Assistant_content_plus_X-GPTHub-Trace
    end
```